JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING
TATHAWADE, PUNE-33

DEPARTMENT OF COMPUTER ENGINEERING

Mini Project

On

" TrafficIQ: AI-Based Adaptive Traffic Signal Control Using Machine Learning"

Submitted by-

Name :- Dange Sanika Vinod

Roll No. :- CS3164

Class and Division: TY - A

Course: Machine Learning

Academic Year 2026-27

(Semester-I)

1. Title :- TrafficIQ: AI-Based Adaptive Traffic Signal Control Using ML

A Dynamic Machine Learning-Based Traffic Signal Optimization System

2. Abstract:-

Traffic congestion is a major urban problem that increases travel time, fuel consumption, and air pollution. Traditional fixed-time traffic signals use predetermined green-light durations without considering variations in traffic volume, often causing unnecessary waiting and long vehicle queues.

TrafficIQ is a machine learning-based system designed to optimize traffic signal timings at a four-way intersection consisting of North, South, East, and West approaches. It uses a Random Forest Regression model to predict traffic demand based on parameters such as vehicle count, traffic density, queue length, average speed, and historical waiting time. The predicted demand is used to allocate green-light durations according to traffic conditions while considering configured timing constraints and signal clearance intervals.

The system includes an interactive traffic simulation and a Streamlit dashboard for displaying vehicle movement, signal timings, traffic statistics, and performance comparisons. TrafficIQ demonstrates how machine learning and adaptive signal optimization can support more efficient traffic management in simulated environments.

3. Introduction :-

Urban traffic congestion affects transportation efficiency and contributes to fuel wastage and environmental pollution. Intersections are common locations for congestion because vehicles travelling in different directions must share limited road space.

Conventional fixed-time traffic signals assign predetermined green-light durations to each approach. These timings may not match actual traffic conditions. For example, a road with a large vehicle queue may require more green time than a road with very few vehicles.

TrafficIQ addresses this limitation by combining machine learning-based traffic demand prediction with adaptive signal timing. The system estimates demand for each direction and adjusts green-time allocation accordingly. An interactive dashboard allows users to observe traffic conditions and compare fixed-time and adaptive signal control.

4. Problem Statement :-

Fixed-time traffic signal systems cannot respond effectively to changing and uneven traffic demand. They may allocate unnecessary green time to less congested roads while heavily congested roads experience longer queues and waiting times.

The problem is to develop a system that predicts traffic demand using relevant traffic parameters, dynamically allocates green-light durations, respects configured signal timing constraints, and visualizes traffic flow at a four-way intersection.

The system should also provide performance indicators to help compare adaptive signal control with conventional fixed-time control under simulated traffic conditions.

5. Objectives:-

The main objectives of TrafficIQ are:

1. To prepare traffic data containing vehicle counts, density, speed, queue length, and waiting time.
2. To develop a Random Forest Regression model for predicting traffic demand.
3. To optimize green-light durations according to predicted traffic demand.
4. To incorporate minimum and maximum green-time constraints and signal clearance intervals.
5. To compare fixed-time and adaptive traffic signal control.
6. To analyze traffic performance using delay, throughput, and other available indicators.

6. Data set and Libraries required :-

➢ Dataset

The project description specifies a synthetic dataset of 6,000 traffic records representing different traffic conditions. The dataset includes the following attributes:

Feature	**Description**
Road	North, South, East, or West
Vehicle Count	Number of observed vehicles
Traffic Density	Vehicles per unit road length
Queue Length	Number of waiting vehicles
Average Speed	Average vehicle speed
Hour and Day	Temporal traffic information
Previous Waiting Time	Waiting time from the previous cycle

The dataset is used to train and evaluate the Machine Learning model. Its actual size and attributes should be verified against the dataset file used in the application.

➢ Libraries and Technologies

- **Python:** Main programming language.
- **Pandas and NumPy:** Data processing and numerical calculations.

- **Scikit-learn:** Random Forest model training and evaluation.
- **Joblib:** Saving and loading the trained model.
- **Plotly:** Data visualization and analytical charts.
- **HTML5 Canvas and JavaScript:** Traffic intersection animation.

7. Approach/ Methodology :-

The Random Forest Regression model is trained using Jupyter Notebook through the following steps.

Step 1: Dataset Preparation

Load `traffic_dataset.csv` using Pandas and prepare the required traffic features and target variable, `traffic_demand`.

Step 2: Data Preprocessing

Handle missing values and encode categorical features while scaling numerical features where required.

Step 3: Data Splitting

Split the dataset into 80% training data and 20% testing data.

Step 4: Model Training

Train a Random Forest Regressor using Scikit-learn to learn patterns in traffic data and predict traffic demand.

Step 5: Model Evaluation

Evaluate the trained model using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score.

Step 6: Model Saving

Save the trained model and required preprocessing components using Joblib for integration into the TrafficIQ application.

8. Implementation:-

The Random Forest Regression model is implemented and trained using Jupyter Notebook.

1. **Dataset Loading:** Load the traffic dataset using Pandas.
2. **Preprocessing:** Clean the data and select input features and target variable.
3. **Model Training:** Split the dataset into 80% training and 20% testing data and train the Random Forest Regressor.
4. **Evaluation:** Evaluate the model using MAE, RMSE, and R^2 score.
5. **Model Saving:** Save the trained model using Joblib for integration into the TrafficIQ application.

Code :-

```
import pandas as pd
import numpy as np
import joblib
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

df = pd.read_csv("traffic_dataset.csv")

features = [
    "road", "vehicle_count", "traffic_density", "queue_length",
    "avg_speed", "hour_of_day", "day_of_week", "is_weekend",
    "prev_waiting_time"
]
target = "traffic_demand"

df = df.dropna(subset=features + [target])
X, y = df[features], df[target]

preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["road"]),
    ("num", StandardScaler(), [c for c in features if c != "road"])
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=120, max_depth=14,
        min_samples_split=4, min_samples_leaf=2,
        random_state=42, n_jobs=-1
    ))
])

model.fit(X_train, y_train)
pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2 Score:", r2_score(y_test, pred))

joblib.dump(model, "traffic_rf_model.joblib")
print("Model trained and saved successfully!")
```

9. Results:-

```
============================================================
 TRAFFICIQ: TRAFFIC DEMAND PREDICTION USING MACHINE LEARNING
============================================================

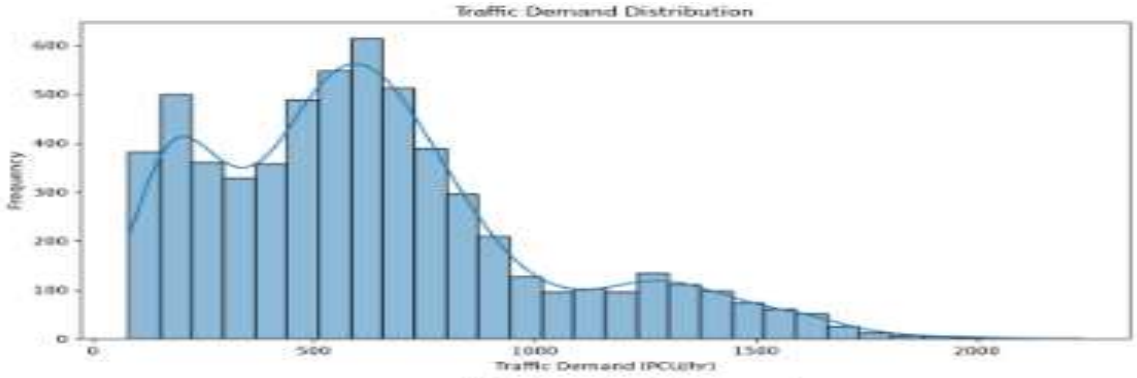
Dataset loaded successfully!
Dataset path: C:\Users\ACER\OneDrive\Desktop\a\traffic_dataset.csv
Dataset shape: (6000, 10)
```

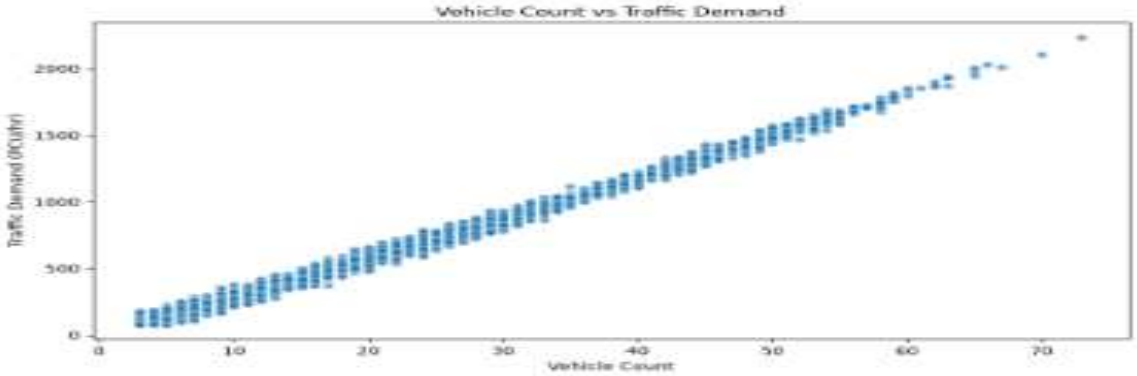
	road	vehicle_count	traffic_density	queue_length	avg_speed	hour_of_day	day_of_week	is_weekend	prev_waiting_time	traffic_demand
0	South	16	33.65	51.98	37.09	6	6	1	17.08	512.84
1	South	22	39.66	118.35	36.27	19	6	1	38.36	648.40
2	East	22	39.94	91.46	40.73	14	3	0	29.82	637.24
3	North	21	37.23	38.61	37.95	10	2	0	16.68	546.76
4	South	49	83.77	233.22	23.53	7	1	0	75.50	1438.97

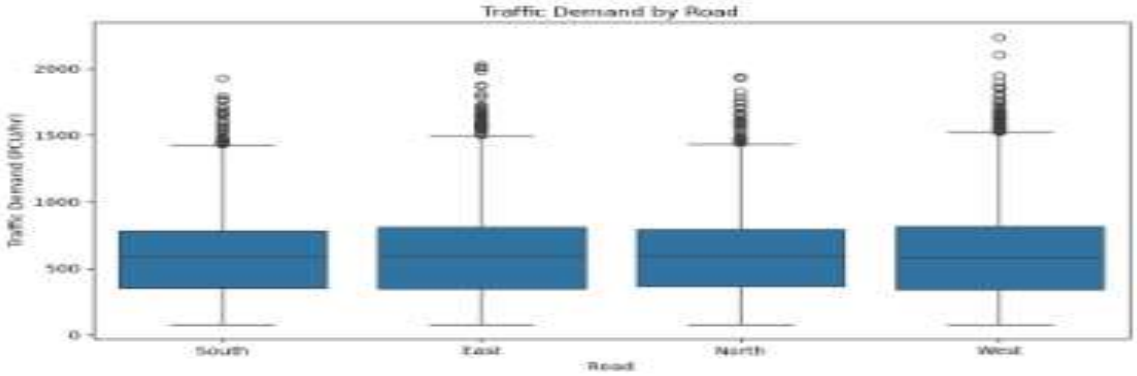
```
Dataset columns:
['road', 'vehicle_count', 'traffic_density', 'queue_length', 'avg_speed', 'hour_of_day', 'day_of_week', 'is_weekend', 'prev_waiting_time', 'traffic_demand']

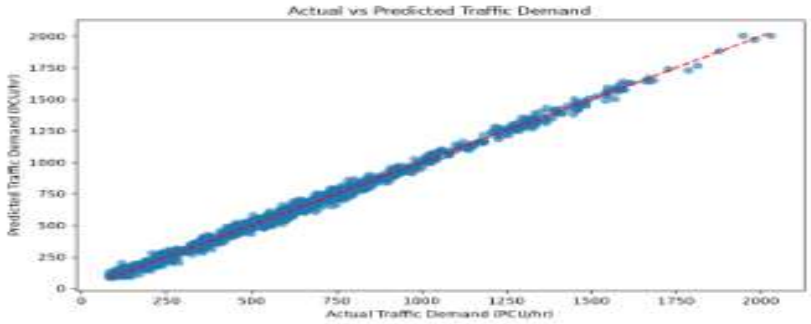
Missing values:
road                0
vehicle_count       0
traffic_density     0
queue_length        0
avg_speed           0
hour_of_day         0
day_of_week         0
is_weekend          0
prev_waiting_time   0
traffic_demand      0
dtype: int64

Usable records: 6000
```

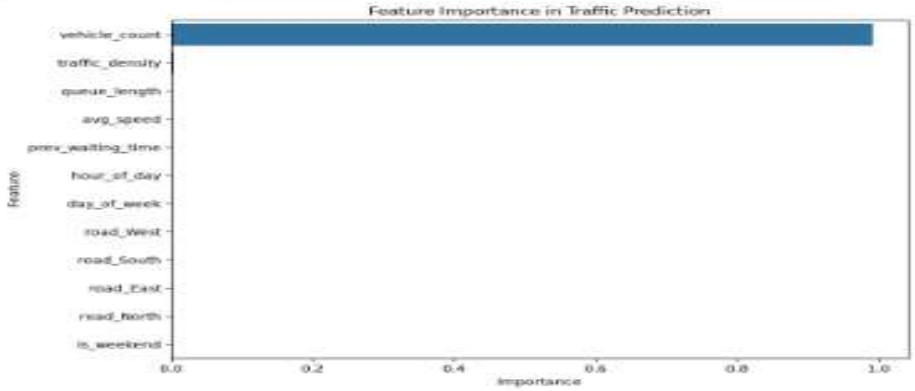








	Feature	Importance
4	vehicle_count	0.941099
5	traffic_density	0.002698
6	queue_length	0.002078
7	avg_speed	0.001653
11	prev_waiting_time	0.000783
8	hour_of_day	0.000359
9	day_of_week	0.000298
3	road_West	0.000073
2	road_South	0.000071
0	road_East	0.000069
1	road_North	0.000065
10	is_weekend	0.000033



	road	vehicle_count	traffic_density	queue_length	avg_speed	hour_of_day	day_of_week	is_weekend	prev_waiting_time
1782	North	17	30.43	62.55	40.02	11	1	0	24.33

10. Conclusion:-

TrafficIQ uses a Random Forest Regression model to predict traffic demand and support adaptive traffic signal optimization. The model is trained and evaluated using Jupyter Notebook and integrated into a Streamlit application. The system demonstrates the potential of machine learning to improve traffic management through demand-based signal timing